

AI ASSIST CODING ASSIGNMENT

Program Name: B. Tech

Course Code: 23CS002PC304

Course Title: AI Assisted Coding

Year/Sem: III/II

Regulation: R23

Student Name: Sanjay Karupothula

Roll No: 2303A52337

Assignment Number: 12.1 / 24

Lab 12: Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms

Lab Objectives

- Apply AI-assisted programming to implement and optimize sorting and searching algorithms.
 - Compare different algorithms in terms of efficiency and use cases.
 - Understand how AI tools can suggest optimized code and complexity improvements.
-

Task 1 – Data Structures: Stack Implementation with AI Assistance

Prompt Used

Generate a Python program that implements a Stack class with push(), pop(), peek(), and is_empty() methods. Include proper error handling for stack underflow and clear docstrings for each method.

Python Implementation

```
class Stack:
    """
    A simple Stack implementation using a Python List.
    """

    def __init__(self):
        """Initialize an empty stack."""
```

```

        self.items = []

    def push(self, element):
        """Push an element onto the stack."""
        self.items.append(element)

    def pop(self):
        """Remove and return the top element of the stack.
        Raises IndexError if the stack is empty (underflow)."""
        if self.is_empty():
            raise IndexError("Stack Underflow: Cannot pop from empty stack.")
        return self.items.pop()

    def peek(self):
        """Return the top element without removing it.
        Raises IndexError if the stack is empty."""
        if self.is_empty():
            raise IndexError("Stack is empty.")
        return self.items[-1]

    def is_empty(self):
        """Return True if the stack is empty, otherwise False."""
        return len(self.items) == 0

```

Testing the Stack

```

stack = Stack()
stack.push(10)
stack.push(20)
print("Top element:", stack.peek())
print("Popped element:", stack.pop())
print("Is stack empty?", stack.is_empty())

```

Explanation

- Implemented Stack using a list.
- Included push, pop, peek, and is_empty methods.
- Added proper error handling for stack underflow using exceptions.
- Included clear docstrings for each method.

Output

Top element: 20 Popped element: 20 Is stack empty? False

Screenshot of Execution

main.py

+

44ehtyyb2

```
1 class Stack:
2     """
3     A simple Stack implementation using a Python list.
4     """
5
6     def __init__(self):
7         """Initialize an empty stack."""
8         self.items = []
9
10    def push(self, element):
11        """Push an element onto the stack."""
12        self.items.append(element)
13
14    def pop(self):
15        """
16        Remove and return the top element of the stack.
17        Raises IndexError if the stack is empty (underflow).
18        """
19        if self.is_empty():
20            raise IndexError("Stack Underflow: Cannot pop from empty stack")
21        return self.items.pop()
22
23    def peek(self):
24        """
25        Return the top element without removing it.
26        Raises IndexError if the stack is empty.
27        """
28        if self.is_empty():
29            raise IndexError("Stack is empty.")
30        return self.items[-1]
31
32    def is_empty(self):
33        """Return True if the stack is empty, otherwise False."""
34        return len(self.items) == 0
35
36
37
```

Output:

Top element: 20
Popped element: 20
Is stack empty? False

main.py

+

44ehtyyb2

```
9
10    def push(self, element):
11        """Push an element onto the stack."""
12        self.items.append(element)
13
14    def pop(self):
15        """
16        Remove and return the top element of the stack.
17        Raises IndexError if the stack is empty (underflow).
18        """
19        if self.is_empty():
20            raise IndexError("Stack Underflow: Cannot pop from empty stack")
21        return self.items.pop()
22
23    def peek(self):
24        """
25        Return the top element without removing it.
26        Raises IndexError if the stack is empty.
27        """
28        if self.is_empty():
29            raise IndexError("Stack is empty.")
30        return self.items[-1]
31
32    def is_empty(self):
33        """Return True if the stack is empty, otherwise False."""
34        return len(self.items) == 0
35
36
37 # Testing the Stack
38 stack = Stack()
39 stack.push(10)
40 stack.push(20)
41
42 print("Top element:", stack.peek())
43 print("Popped element:", stack.pop())
44 print("Is stack empty?", stack.is_empty())
```

Output:

Top element: 20
Popped element: 20
Is stack empty? False

Task 2 – Algorithms: Linear vs Binary Search Analysis

Prompt Used

Generate Python implementations of `linear_search(arr, target)` and `binary_search(arr, target)`. Include docstrings explaining working principle and perform testing with different input sizes.

Python Implementation

```
def linear_search(arr, target):
    """
    Perform Linear Search.
    Checks each element sequentially until target is found.
    Time Complexity: O(n)
    """
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

def binary_search(arr, target):
    """
    Perform Binary Search.
    Works only on sorted arrays. Divides search space in half each step.
    Time Complexity: O(log n)
    """
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1

# Testing
arr_small = [1, 3, 5, 7, 9]
arr_large = list(range(1, 10001))
```

```
print("Linear Search (small array):", linear_search(arr_small, 7))
print("Binary Search (small array):", binary_search(arr_small, 7))

print("Linear Search (large array):", linear_search(arr_large, 9999))
print("Binary Search (large array):", binary_search(arr_large, 9999))
```

Explanation

- Linear Search checks elements sequentially.
- Binary Search divides the sorted array into halves.
- Binary Search is more efficient for large sorted datasets.
- Complexity Comparison:
 - Linear Search: $O(n)$
 - Binary Search: $O(\log n)$

Output

Linear Search (small array): 3 Binary Search (small array): 3 Linear Search (large array): 9998 Binary Search (large array): 9998

Screenshot of Execution

main.py

44ehtyib2

```
1 def linear_search(arr, target):
2     """
3     Perform Linear Search.
4     Checks each element sequentially until target is found.
5     Time Complexity: O(n)
6     """
7     for i in range(len(arr)):
8         if arr[i] == target:
9             return i
10    return -1
11 def binary_search(arr, target):
12     """
13     Perform Binary Search.
14     Works only on sorted arrays.
15     Divides search space in half each step.
16     Time Complexity: O(log n)
17     """
18    low = 0
19    high = len(arr) - 1
20
21    while low <= high:
22        mid = (low + high) // 2
23
24        if arr[mid] == target:
25            return mid
26        elif arr[mid] < target:
27            low = mid + 1
28        else:
29            high = mid - 1
30
31    return -1
32
33 # Testing
34 arr_small = [1, 3, 5, 7, 9]
35 arr_large = list(range(1, 10001))
36
37
```

Output:

Linear Search (small array): 3
Binary Search (small array): 3
Linear Search (large array): 9998
Binary Search (large array): 9998

OneCompiler

Q ⚙ Pri

main.py

44ehtyib2

```
13 Perform Binary Search.
14 Works only on sorted arrays.
15 Divides search space in half each step.
16 Time Complexity: O(log n)
17 """
18 low = 0
19 high = len(arr) - 1
20
21 while low <= high:
22     mid = (low + high) // 2
23
24     if arr[mid] == target:
25         return mid
26     elif arr[mid] < target:
27         low = mid + 1
28     else:
29         high = mid - 1
30
31 return -1
32
33 # Testing
34 arr_small = [1, 3, 5, 7, 9]
35 arr_large = list(range(1, 10001))
36
37 print("Linear Search (small array):", linear_search(arr_small, 7))
38 print("Binary Search (small array):", binary_search(arr_small, 7))
39
40 print("Linear Search (large array):", linear_search(arr_large, 9999))
41 print("Binary Search (large array):", binary_search(arr_large, 9999))
42
43
44
45
46
```

Output:

Linear Search (small array): 3
Binary Search (small array): 3
Linear Search (large array): 9998
Binary Search (large array): 9998

Comparison and Analysis

- Linear Search is simple and works on unsorted data.
 - Binary Search requires sorted data but is significantly faster.
 - AI assistance helped generate structured code, docstrings, and complexity analysis quickly.
-

Conclusion

This lab demonstrated how AI tools can assist in implementing data structures and algorithms efficiently. The Stack implementation included proper error handling and documentation. The comparison between Linear and Binary Search highlighted differences in performance and complexity, showing how optimized algorithms improve efficiency for large datasets.